

Taller de Minería de Datos

Ejercicios 1 a 5 — Tipos de datos, matrices, distancias y PCA

17 de septiembre de 2026

Ejercicio 1: clasificación de tipos de datos

Para cada fuente de datos se indica: **(a)** tipo de dato, **(b)** la razón, y **(c)** el paso mínimo de preprocesamiento para convertirla en un tibble analizable.

1. Microdatos GEIH del DANE (.csv, 300.000 filas)

- **(a)** Estructurada
- **(b)** Es un CSV con filas y columnas fijas (variables como edad, ingreso, ocupación), con un esquema tabular consistente en todas las filas.
- **(c)** Solo se necesita leer el archivo y tipar las columnas correctamente:

```
geih <- read_csv("microdatos_geih.csv") |>
  as_tibble() |>
  mutate(across(where(is.character), as.factor))
```

2. Respuesta JSON de la API de datos.gov.co (calidad del aire)

- **(a)** Semi-estructurada
- **(b)** El JSON tiene un esquema (llaves/valores), pero puede tener anidamiento o campos que no aparecen en todos los registros, a diferencia de un CSV plano.
- **(c)** Aplanar (*flatten*) el JSON a formato tabular:

```
aire <- fromJSON("https://www.datos.gov.co/resource/xxxx.json") |>
  as_tibble() |>
  unnest_wider(alguna_columna_anidada)
```

3. Grabaciones de audio de audiencias judiciales

- **(a)** No estructurada
- **(b)** Es una señal de audio continua, sin ningún esquema de filas/columnas ni etiquetas.
- **(c)** Requiere transcripción de voz a texto (*speech-to-text*) y luego estructurar esa salida en un tibble:

```
transcripcion <- transcribir_audio("audiencia_001.mp3")

audiencias <- tibble(
  archivo = "audiencia_001.mp3",
  texto   = transcripcion$texto,
  minuto  = transcripcion$timestamps
)
```

4. Factura electrónica DIAN (formato XML)

- **(a)** Semi-estructurada
- **(b)** El XML tiene jerarquía definida por etiquetas (emisor, receptor, ítems, impuestos), pero es anidado y con secciones repetibles, no una tabla plana.
- **(c)** Parsear el XML y extraer los nodos relevantes a columnas de un tibble:

```
items <- read_xml("factura.xml") |>
  xml_find_all(".*//cac:InvoiceLine") |>
  purrr::map_df(~ tibble(
    descripcion = xml_text(xml_find_first(.x, ".*//cbc:Description")),
    cantidad    = as.numeric(xml_text(xml_find_first(.x,
".//cbc:InvoicedQuantity"))),
    valor       = as.numeric(xml_text(xml_find_first(.x,
".//cbc:LineExtensionAmount"))))
  ))
```

5. Tabla HTML de Wikipedia scrapeada con `rvest`

- **(a)** Estructurada (una vez extraída); la página HTML completa de origen es semi-estructurada.
- **(b)** El HTML completo mezcla contenido, marcado y estilos, pero la tabla `<table>` que se extrae ya tiene filas y columnas bien definidas.
- **(c)** Extraer la tabla y convertirla a tibble:

```
tabla <- read_html("https://es.wikipedia.org/wiki/...") |>
  html_element("table.wikitable") |>
  html_table() |>
  as_tibble()
```

Ejercicio 2: JSON anidado a DataFrame

Fuente: <https://jsonplaceholder.typicode.com/users>

1. Estructura del objeto (`str(raw)`)

`raw` es un `data.frame` de 10 observaciones y 8 variables. Dos de esas variables son **columnas-lista anidadas**:

- `address`: a su vez un `data.frame` con `street`, `suite`, `city`, `zipcode` y una columna anidada `geo` (con `lat`, `lng`).
- `company`: un `data.frame` con `name`, `catchPhrase`, `bs`.

2. Tibble aplanado (una fila por usuario)

```
usuarios <- raw |>
  as_tibble() |>
  unnest_wider(address, names_sep = "_") |>
  unnest_wider(company, names_sep = "_") |>
  unnest_wider(address_geo, names_sep = "_") |>
  select(
    name,
    email,
    address.city = address_city,
    address.geo.lat = address_geo_lat,
    address.geo.lng = address_geo_lng,
    company.name = company_name
  )
```

Resultado (10 filas, 6 columnas):

name	email	address.city	geo.lat	geo.lng	company.name
Leanne Graham	Sincere@april.biz	Gwenborough	-37.3159	81.1496	Romaguera-Crona
Ervin Howell	Shanna@melissa.tv	Wisokyburgh	-43.9509	-34.4618	Deckow-Crist
Clementine Bauch	Nathan@yesenia.net	McKenziehaven	-68.6102	-47.0653	Romaguera-Jacobson
Patricia Lebsack	Julianne.OConner@kory.org	South Elvis	29.4572	-164.2990	Robel-Corkery
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview	-31.8129	62.5342	Keebler LLC
Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy	-71.4197	71.7478	Considine-Lockman
Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth	24.8918	21.8984	Johns Group
Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview	-14.3990	-120.7677	Abernathy Group
Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury	24.6463	-168.8889	Yost and Sons
Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury	-38.2386	57.2232	Hoeger LLC

3. ¿Qué tipo de fuente es y por qué `fromJSON` no produce un tibble plano?

Es una fuente **semi-estructurada**: el JSON tiene un esquema (llaves consistentes en todos los registros), pero con anidamiento (objetos dentro de objetos: `address` dentro de cada usuario, `geo` dentro de `address`). `fromJSON()` convierte el JSON a listas/data.frames anidados porque un `data.frame` de R es inherentemente plano (dos dimensiones); no puede representar de forma nativa una columna cuyo valor es a su vez otra tabla. Por eso `jsonlite` deja esas columnas como data.frames anidados, y es tarea del analista “aplanarlos” con `unnest_wider()`.

Ejercicio 3: matriz de diseño

Variables usadas: `age`, `fare`, `sibsp`, `parch` (con `na.omit()`), sobre `titanic.csv`.

1. Matriz X centrada y escalada

```
vars <- titanic |> select(age, fare, sibsp, parch) |> na.omit()
X <- scale(as.matrix(vars))
```

- **Dimensiones:** 1045 filas × 4 columnas
- **Memoria ocupada:** 99.9 Kb

2. Cálculo de $X^T X$

Matriz $X^T X$ (filas/columnas en orden: `age`, `fare`, `sibsp`, `parch`):

	age	fare	sibsp	parch
age	1044.00	186.60	-253.84	-156.85
fare	186.60	1044.00	147.40	226.26
sibsp	-253.84	147.40	1044.00	390.76
parch	-156.85	226.26	390.76	1044.00

Interpretación de la diagonal: cuando X está escalada (media 0, varianza 1 por columna), $X^T X / (n-1)$ es la **matriz de correlación** de las variables. La diagonal de $X^T X$ equivale a $(n-1)$ veces la varianza de cada variable escalada, que es 1, por lo que la diagonal es simplemente $(n-1) = 1044$ en cada entrada (la suma de cuadrados de una variable estandarizada).

3. Versión dispersa

```
X_sparse <- Matrix(X, sparse = TRUE)
```

- Tamaño denso: **99.9 Kb**
- Tamaño disperso: **116.1 Kb**

¿Tiene sentido usar formato disperso aquí? No. Al escalar (centrar y dividir por la desviación estándar) casi ningún valor queda en cero, así que la matriz queda densa. El formato disperso solo ahorra memoria cuando la mayoría de las entradas son cero; en este caso ocupa incluso *más* memoria que la versión densa, por el overhead de guardar los índices de fila/columna.

4. Estructura para relaciones familiares

Para modelar relaciones familiares entre pasajeros (`sibsp`, `parch`) lo natural es un **grafo** (o una matriz de adyacencia dispersa), no una matriz numérica densa como X . Cada pasajero es un nodo y cada relación familiar (hermano/cónyuge, padre/hijo) es una arista. Como la mayoría de los pasajeros no tienen relación familiar entre sí, la matriz de adyacencia sería muy dispersa (mayormente ceros), así que ahí sí conviene una matriz dispersa (`Matrix::sparseMatrix`) o una estructura de grafo con el paquete `igraph`.

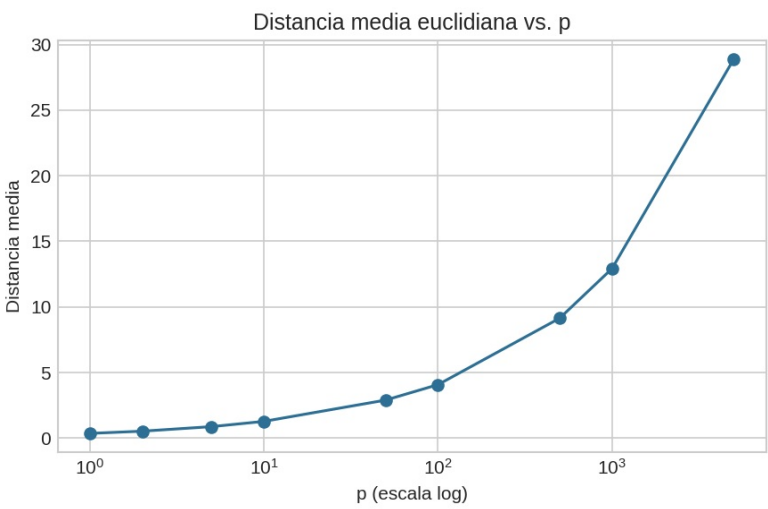
Ejercicio 4: concentración de distancias

Para cada $p \in \{1, 2, 5, 10, 50, 100, 500, 1000, 5000\}$ (con `set.seed(42)`), se generaron 500 puntos uniformes en $[0,1]^p$ y se calcularon 200 distancias entre pares aleatorios.

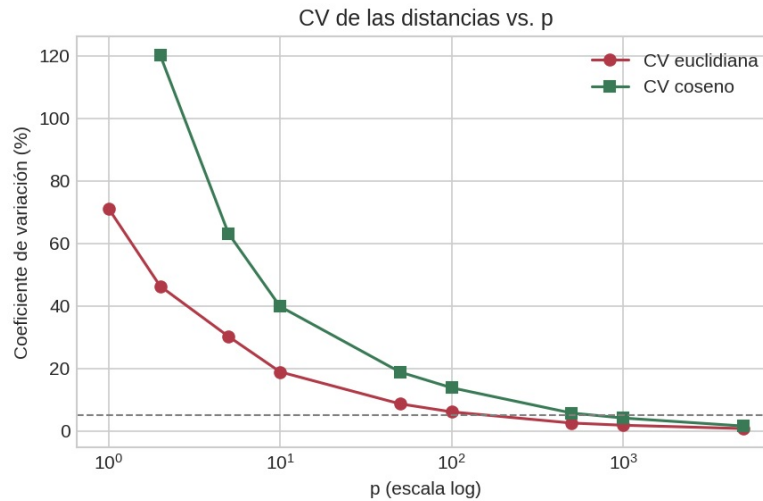
1. Distancia media y CV por valor de p

p	Distancia media (euclidiana)	CV euclidiana (%)	Distancia media (coseno)	CV coseno (%)
1	0.357	71.21	0.000	NaN
2	0.527	46.34	0.153	120.26
5	0.873	30.26	0.228	63.20
10	1.274	18.92	0.243	39.83
50	2.895	8.70	0.254	18.86
100	4.072	6.12	0.246	13.83
500	9.137	2.55	0.251	5.75
1000	12.922	1.85	0.251	4.15
5000	28.884	0.78	0.250	1.59

2. Gráficas



Distancia media euclidiana vs. p



Coeficiente de variación (euclidiana y coseno) vs. p

3. ¿A partir de qué p el CV cae bajo 5%?

Con la semilla `set.seed(42)`, el CV euclidiano cae por debajo del 5% **a partir de $p = 500$** .

Implicación para K-NN: cuando el CV de las distancias tiende a 0, todos los puntos quedan aproximadamente equidistantes entre sí. Esto es grave para K-NN porque el algoritmo depende de que existan vecinos claramente “más cercanos” que otros; si todas las distancias son casi iguales, el concepto de “vecino más cercano” pierde sentido y K-NN deja de ser informativo — es la conocida **maldición de la dimensionalidad**.

4. Distancia coseno vs. euclidiana

En esta corrida, la distancia coseno se concentra **más lento** que la euclidiana: para el mismo p , el CV coseno es siempre más alto (p. ej., en $p = 500$: euclidiana = 2.55% vs. coseno = 5.75%). El valor `NaN` en $p = 1$ ocurre porque con un solo escalar la “distancia coseno” queda indefinida (dos números positivos escalares son siempre perfectamente “paralelos”, dando denominador o numerador degenerado). Vale la pena reportar esto como una observación empírica, ya que contrasta con la intuición de que ambas métricas se concentran a tasas similares.

Ejercicio 5: PCA y reducción de dimensionalidad

Variables numéricas escaladas: `age`, `fare`, `sibsp`, `parch`, `pclass`.

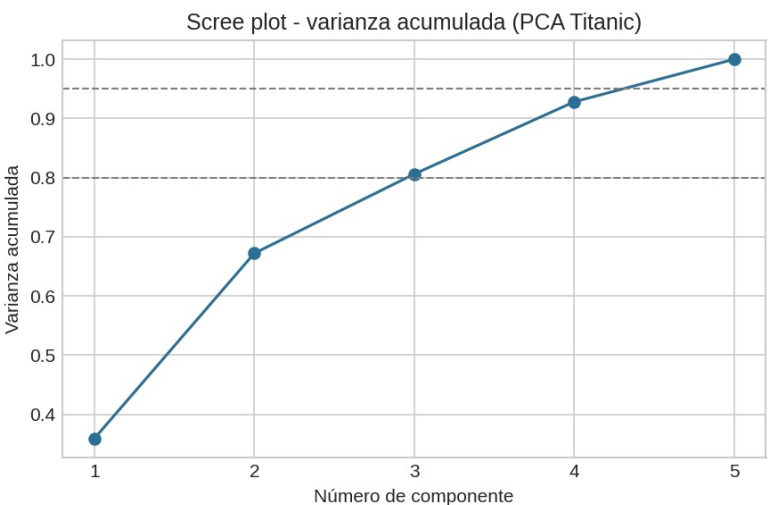
1. Componentes necesarios

```
pca <- prcomp(X5, scale. = TRUE)
```

Componente	Varianza acumulada
PC1	35.9%
PC2	67.2%
PC3	80.6%
PC4	92.8%
PC5	100.0%

- Componentes para $\geq 80\%$ de varianza: 3
- Componentes para $\geq 95\%$ de varianza: 5 (prácticamente todas)

2. Scree plot



Scree plot — varianza acumulada

3. Loadings de PC1 y PC2

Variable	PC1	PC2
age	-0.4918	0.3124
fare	-0.5631	-0.3303
sibsp	0.0875	-0.6311
parch	-0.0047	-0.6277
pclass	0.6583	0.0302

Interpretación:

- **PC1** está dominado por `pclass` (0.658) y `fare` (-0.563), con signos opuestos — tarifa alta se asocia con `pclass` numéricamente bajo (primera clase). Se puede

interpretar como un eje de **estatus socioeconómico**.

- **PC2** está dominado por `sibsp` (-0.631) y `parch` (-0.628) — interpretable como un eje de **tamaño del grupo familiar a bordo**.

4. K-Means: X original vs. las 2 primeras componentes

```
km_original <- kmeans(scale(X5), centers = 2, nstart = 25)
km_pca      <- kmeans(pca$x[, 1:2], centers = 2, nstart = 25)
```

Cluster vs. `survived` — X original:

Cluster	No sobrevivió (0)	Sobrevivió (1)
1	441	280
2	142	182

Cluster vs. `survived` — 2 primeras PCs:

Cluster	No sobrevivió (0)	Sobrevivió (1)
1	145	184
2	438	278

Interpretación: los dos agrupamientos son prácticamente equivalentes (solo se invirtieron las etiquetas de cluster 1 y 2, algo normal en K-Means). En ninguna de las dos representaciones los clusters separan limpiamente la supervivencia: el cluster más grande (~720 personas) mezcla sobrevivientes y no sobrevivientes en proporciones similares en ambos casos. Esto es coherente con que la supervivencia dependía fuertemente de la variable `sex`, que no se incluyó en este análisis (solo entra indirectamente a través de `pclass`). Dado que con solo 5 variables las 2 primeras componentes ya retienen el 67% de la varianza, no sorprende que K-Means sobre las 2 PCs dé resultados casi idénticos a K-Means sobre los datos originales.